

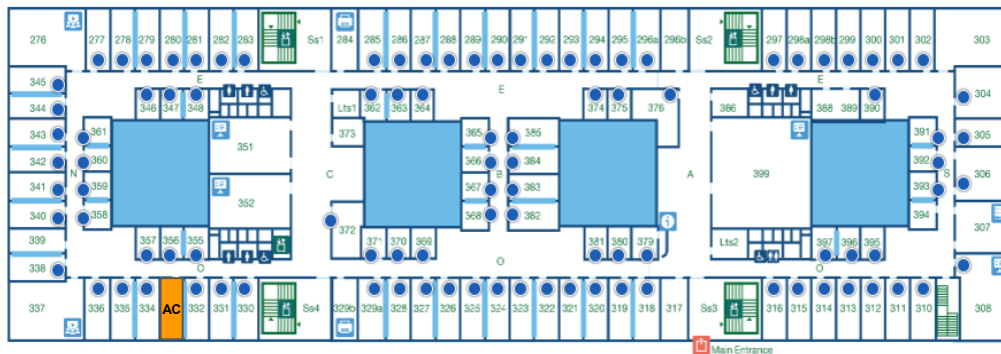
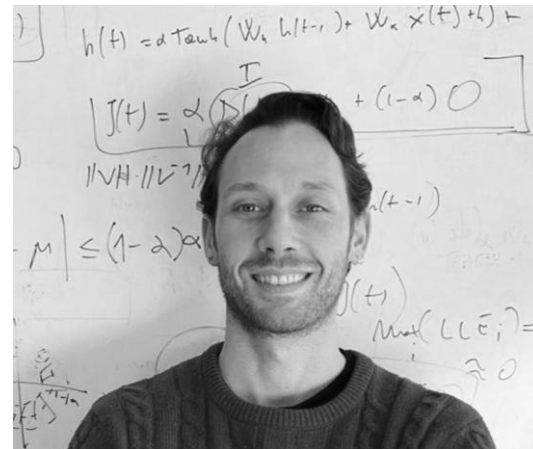
Laboratorio I

a.a. 2025/2026

JavaScript Recap

Docenti – Andrea Ceni

Email	andrea.ceni@unipi.it
Web	https://www.unipi.it/ateneo/organizzazione/persone/andrea-ceni-173990/
Ufficio	Stanza 333, Dipartimento di Informatica
Ricevimento	Martedì 17:30-18:30



Tipi di dato e valori letterali

JavaScript consente di denotare **valori letterali** di determinati tipi:

- **Boolean:** `true`, `false`
- **Undefined:** `undefined` ($\rightarrow$ non è stato definito nessun valore)
- **Null:** `null` ($\rightarrow$ so che non c'è un valore)
- **Number:** `1`, `5`, `-53.38`, `12.3e4` (numeri “normali”, $-(10^{53}-1) \leq n \leq 10^{53}-1$)
- **BigInt:** `90071992547434344169871610992n` (interi a precisione infinita)
- **String:** `“che bel castello”`, `“हिन्दू”`, `‘hello’`, ``x vale ${x}`` (sequenze di caratteri)
- **Object:** `{ nome: “Pino”, età: 19 }` (mappe da chiavi a valori - anche dizionari)
- **Array:** `[ 1, 5, 8, 12, 21, 33 ]`, `[ 1, “ciao!”, { x: 12, y:22 }, [ “banana” ] ]` (liste con indici numerici)
- **Function:** `x=>2*x`, `(a,b) => a+b`, `x=> { if (x>0) return x; else return -x }` (funzioni)

Comando condizionale - if

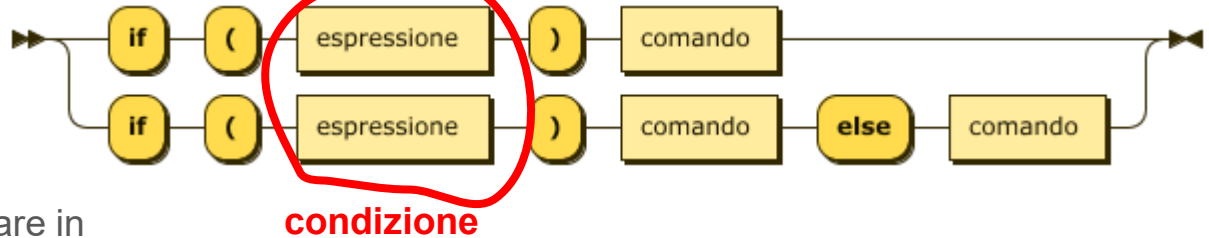


- Comando `if..else`

- Esegue primo *comando* solo se *espressione* vale `true`, altrimenti esegue secondo *comando*
- *espressione* ha di solito un valore Booleano - può essere qualsiasi tipo di dato (cast automatico).

```
comando_if ::=  
if ( espressione ) comando |  
if ( espressione ) comando else comando
```

```
if (m>0)  
| console.log(`${m} minuti`);
```

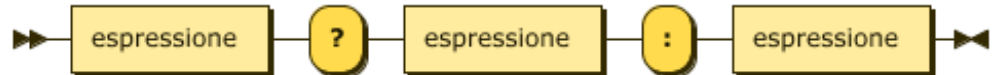


- Operatore logico `?:`

- Simile al `if` però da usare in espressioni

```
console.log(m>0?`${m} minutes`:``);
```

```
exp_logica_terziaria ::= espressione ? espressione : espressione
```

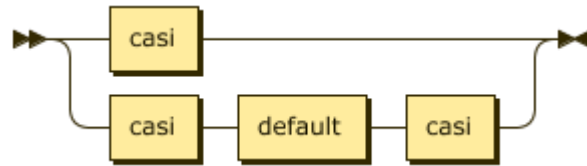


Comando condizionale - switch

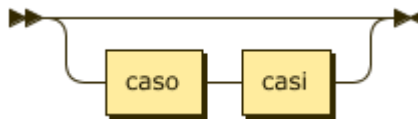
comando_switch:



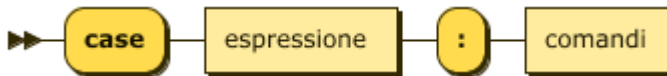
elenco_casi:



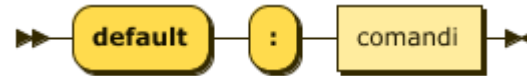
casi:



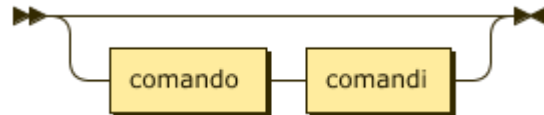
caso:



default:



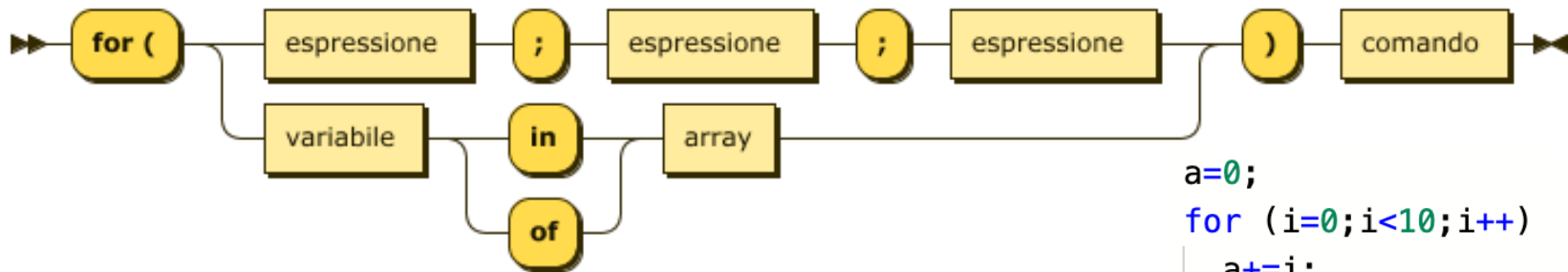
comandi:



```
comando_switch ::=  
    'switch' '(' espressione ')' '{' elenco_casi '}'  
elenco_casi ::= casi | casi default casi  
casi ::= | caso casi  
caso ::= 'case' espressione ':' comandi  
default ::= 'default' ':' comandi  
comandi ::= | comando comandi
```

```
switch (n) {  
    case 1: /* fa qualcosa */  
        break;  
    case 2: /* fa qualcos'altro */  
        break;  
    default: /* disperati */  
        break  
}
```

Comando iterativo - for



```
comando_for ::= 'for' '('  
  ( espressione ';' espressione ';' espressione  
  | variabile ( 'in' | 'of' ) array )  
  ')' comando
```

```
a=0;  
for (i=0;i<10;i++)  
  a+=i;
```

```
let studenti=['Matt','Alice','Bob'];  
for (i in studenti)  
  console.log(studenti[i]);
```

- Permette di ripetere un comando o set di comandi un numero di volte
- Di solito usato quando si conosce il numero di iterazioni
- Esempio: somma dei primi n numeri naturali

```
let studenti=['Matt','Alice','Bob'];  
for (s of studenti)  
  console.log(s);
```

Comando iterativo - while, do while



- Un altro tipo di comando per ripetere istruzioni
- Esegue *comando* fino a quando *espressione* diventa *false*
- *espressione* ha di solito un valore Booleano
 - Può essere qualsiasi tipo di dato (cast automatico).
- Di solito usato quando non si conosce il numero di iterazioni
- Esempio: sommare numeri in input finché l'utente inserisce -1

```
comando_while ::= while ( espressione ) comando
```



```
comando_do ::= do comando while ( espressione )
```



```
while(condizione){  
    //comandi  
}  
  
do{  
    //comandi  
} while (condizione);
```

Funzioni: valori & letterali

Abbiamo visto come le *funzioni* siano uno dei tipi base di JavaScript

Come tali, possono essere denotate da costanti letterali

(ovvero: è possibile esprimere un valore di tipo funzione scrivendolo nel testo di un programma)

Un valore di tipo funzione è pur sempre un valore

(quindi, può essere usato nelle espressioni, può essere il risultato di una espressione, può essere assegnato a una variabile, inserito in un array, ecc.)

Si può esprimere una costante letterale di tipo funzione in due modi:

1. Con la notazione “freccia” $\Rightarrow$ (anche detta *lambda espressione*)
2. Con la parola chiave **function**

Funzioni: dichiarazione

Con una notazione molto comune, si può scrivere

```
function f(a,b) { return a+b }
```

Al posto di

```
var f = (a,b)=> { return a+b }
```

Dichiara una funzione di nome *f* con i parametri formali e il corpo indicati.

Equivalente a dichiarare una variabile (notate il var: non let, non const) di nome *f* e assegnare il valore-funzione corrispondente.

Altri linguaggi consentono solo la forma dichiarazione, e le funzioni non sono valori come gli altri ma “cose a parte” del linguaggio

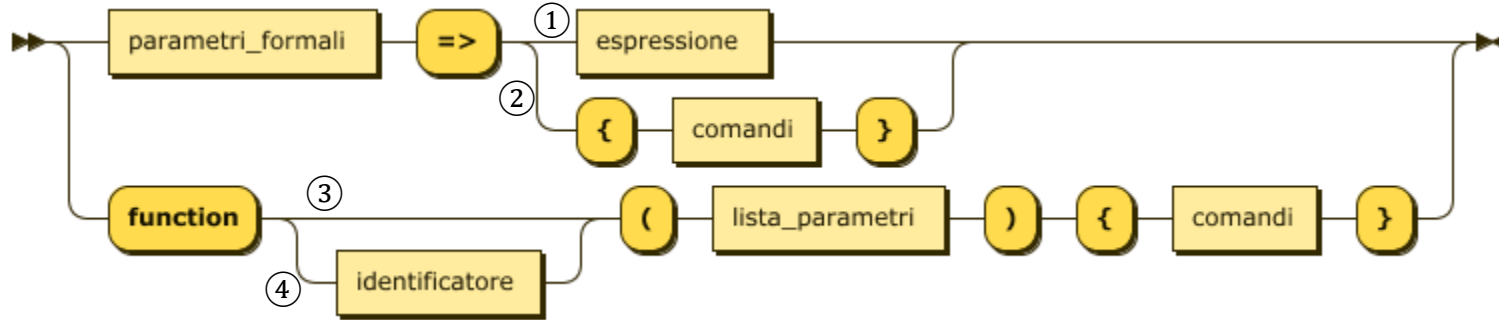
E' molto comune che un programma di grandi dimensioni consti quasi esclusivamente di moltissime dichiarazioni di tipo **function nome(...)** {...}, seguite magari da un solo comando che fa “partire” la computazione.

Funzioni: costanti letterali

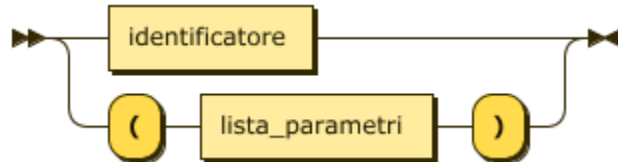
```
letterale_funzione ::=  
    parametri_formali '=>' espressione  
    | parametri_formali '=>' '{' comandi '}'  
    | 'function' '(' lista_parametri ')' '{' comandi '}'  
    | 'function' identificatore '(' lista_parametri ')' '{' comandi '}'
```

```
parametri_formali ::= identificatore | '(' lista_parametri ')'  
lista_parametri ::= | identificatore | identificatore ',' lista_parametri
```

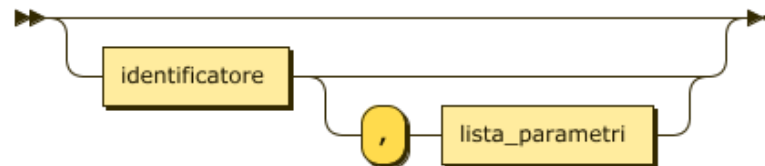
letterale_funzione:



parametri_formali:



lista_parametri:



Sembra complesso ma sono solo due forme (con `=>` e con `function`), ciascuna con due varianti.

① ② ③ ④

Funzioni: costanti letterali

Alcuni esempi...

① `let raddoppia = x => 2*x`

① `var somma = (a,b) => a+b`

② `let prossimopari = (x) => { if (x%2==0) return x+2 else return x+1 }`

① `const cinque = () => 5`

③ `somma = function(a,b) { return a+b }`

④ `f = function somma(a,b) { return a+b }`

Le parti azzurre sono **costanti letterali di tipo funzione**, e dunque denotano dei **valori**, esattamente come 5, 3.1419826 o “pippo”

Cosa accade durante una chiamata

let f = (x => 2*x)

corpo

parametro attuale
o argomento

parametro formale

f(3) → 2*3 → 6

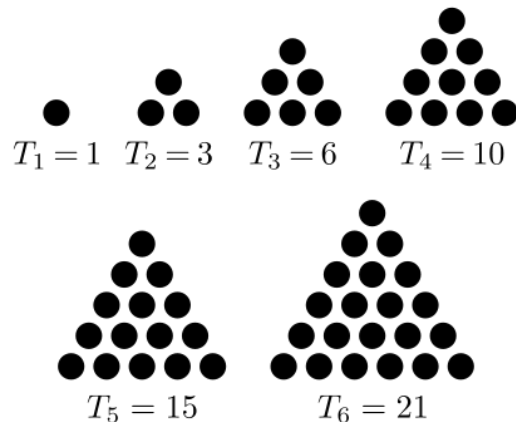
*Il parametro attuale viene sostituito al parametro formale nel corpo della funzione, e l'espressione risultante viene valutata.
Il risultato di questa valutazione è il valore restituito dalla chiamata*

Casi strani ma corretti
Non è necessario dare
un nome al letterale
funzione

(x=>2*x) (1+2) → 6
Valore di tipo funzione Operatore di applicazione (con argomento 3) Risultato

Funzioni ricorsive

- Funzioni definite in termini di loro stesse (ovvero che contengono almeno una chiamata a loro stesse)
- Utilizzano lo stack di record di attivazione (pila)
- Simili al principio di induzione
 - Passo di base - calcolo diretto
 - Passo induttivo - regola per calcolo ricorsivo
- Ricorsione in coda: ultima operazione è la chiamata ricorsiva (più efficiente)



```
function triangle(n){  
  if(n==1)  
    return 1;  
  return n+triangle(n-1);  
}  
  
function triangleCoda(n, s=0){  
  if(n<1)  
    return s;  
  return triangleCoda(n-1,s+n);  
}
```

Dichiarazione e inizializzazione

La **dichiarazione** di una variabile consiste nel definire il suo **nome** (identificatore)

L'**inizializzazione** di una variabile consiste nell'assegnarle un valore per la prima volta

Ricordate: una **dichiarazione** di funzione è un modo diverso (ma equivalente) di scrivere una **dichiarazione** di variabile, e **inizializzarla** con un valore di tipo funzione

<pre>function nome(a₁, ... a_n) { /* corpo */ }</pre>		<pre>var nome = (a₁, ... a_n) => { /* corpo */ }</pre>
--	---	--

Visibilità delle dichiarazioni

Un nome dichiarato è **visibile** in certe parti del programma, più o meno ampie a seconda di *dove* è stato dichiarato e di *quale tipo* di dichiarazione viene usata.

JavaScript prevede tre **scope** o **ambiti di visibilità**:

- Scope globale
- Scope di funzione
- Scope di blocco

```
console.log("benvenuto")
n = 1

function map(f,a) {
  let r=[]
  for (var i=0;i<a.length;i++) {
    r.push(f(n,a[i]))
    n= 1-n
  }
  return r
}

function mul(a,b) { return a*b }

const aa = [4,7,12,3,22,1,6]
var t=map(mul,aa)
console.log(t)
```

Visibilità delle dichiarazioni

Le **dichiarazioni** fatte fuori da qualunque funzione avvengono nello **scope globale**.

Queste dichiarazioni sono **visibili** in tutto il programma, sia fuori che dentro le funzioni o i blocchi.

Le variabili non dichiarate dal programma, ma usate senza dichiarazione, si intendono implicitamente dichiarate nello scope globale.

- Scope **globale**
- Scope di funzione
- Scope di blocco

```
console.log("benvenuto")
```

```
n = 1
```

```
function map(f,a) {  
  let r=[]  
  for (var i=0;i<a.length;i++) {  
    r.push(f(n,a[i]))  
    n= 1-n  
  }  
  return r  
}
```

```
function mul(a,b) { return a*b }
```

```
const aa = [4,7,12,3,22,1,6]
```

```
var t=map(mul,aa)
```

```
console.log(t)
```


Visibilità delle dichiarazioni

Ogni funzione definisce il proprio **scope di funzione**.

I parametri formali si intendono dichiarati nello scope di funzione (ma il nome della funzione no: è nello scope globale!).

Le **dichiarazioni** fatte all'interno di una funzione con **var** sono **visibili** nello scope di quella funzione, ma non fuori.

- Scope globale
- Scope di **funzione**
- Scope di blocco

```
console.log("benvenuto")  
n = 1
```

```
function map(f,a) {  
  let r=[]  
  for (var i=0;i<a.length;i++) {  
    r.push(f(n,a[i]))  
    n= 1-n  
  }  
  return r  
}
```

```
function mul(a,b) { return a*b }
```

```
const aa = [4,7,12,3,22,1,6]  
var t=map(mul,aa)  
console.log(t)
```

Visibilità delle dichiarazioni

Ogni blocco (0 o più comandi racchiusi fra {...}) definisce il proprio **scope di blocco**.
In particolare, il corpo di una funzione è un blocco.
Un blocco può essere contenuto in un altro blocco.
Le dichiarazioni all'interno di un `for` si considerano fatte nel suo blocco¹.

Le **dichiarazioni** fatte all'interno di un blocco con `let` e `const` sono **visibili** nello scope di quel blocco, ma non fuori.

- Scope globale
- Scope di funzione
- Scope di **blocco**

```
console.log("benvenuto")  
n = 1
```

```
function map(f,a) {  
  let r=[]  
  for (var i=0;i<a.length;i++) {  
    r.push(f(n,a[i]))  
    n= 1-n  
  }  
  return r  
}
```

```
function mul(a,b) { return a*b }
```

```
const aa = [4,7,12,3,22,1,6]  
var t=map(mul,aa)  
console.log(t)
```

¹ anche `if` e `while`, ma uso rarissimo!

var, let, const e function

Riassumendo:

1. JavaScript ha scope **globale**, di **funzione**, e di **blocco**
2. C'è un solo scope **globale**, ma gli scope di **funzione** e di **blocco** possono essere annidati uno nell'altro - quando si dichiarano delle variabili con lo stesso nome si utilizza lo *shadowing*
3. Le dichiarazioni con **let** e **const** hanno scope di **blocco**, o di **funzione**, o **globale** (vale lo scope più interno in cui sono scritte)
4. Le dichiarazioni con **var** hanno scope di **funzione** o **globale** (vale lo scope più interno in cui sono scritte)
5. Le variabili non dichiarate hanno scope **globale**
6. Le dichiarazioni con **function** si comportano come **var**, ma si “portano dietro” l'inizializzazione in caso di *hoisting*

Closure

Function è un tipo di dato come tutti gli altri in Javascript

Possiamo usare una funzione in un'espressione o comando

Passare una funzione come parametro a un'altra funzione

Restituire una funzione da un'altra funzione

Closure: Lo scope attivo durante la dichiarazione della funzione rimane visibile alla funzione restituita anche se la funzione padre finisce

```
function f2(f){  
  return x => 2*f(x);  
}  
  
var myF=f2(Math.sqrt);  
  
myF(9); // -> 6
```

Oggetti (o *dizionari*)

Un **oggetto** (o dizionario) è una *mappa* da chiavi a valori

Qui *mappa* è usato nel senso matematico: in pratica, è una funzione matematica il cui dominio è l'insieme di tutte le chiavi (in pratica¹: stringhe e numeri), e il codominio è l'insieme di tutti i valori possibili in JavaScript.

Potremmo anche dire che un oggetto è un insieme di coppie (chiave, valore).

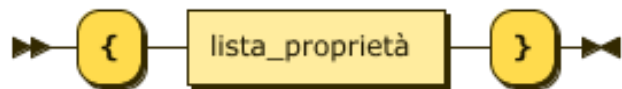
Per esempio, un oggetto *pippo* potrebbe essere indicato in stile matematico con

$$pippo = \{ ("passione", "Fondamenti"), ("età", 35), ("capelli", true) \}$$

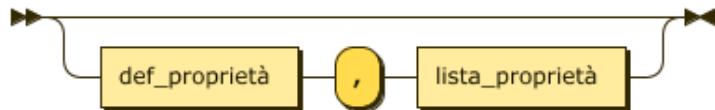
¹ In realtà, anche i famigerati Symbol possono fungere da chiavi: ma è un uso avanzato che per il momento, e per molto tempo ancora, non ci interessa esplorare.

Costanti letterali di tipo oggetto - Sintassi

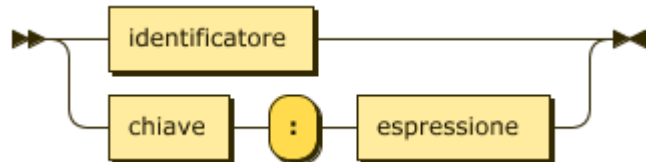
letterale_oggetto:



lista_proprietà:

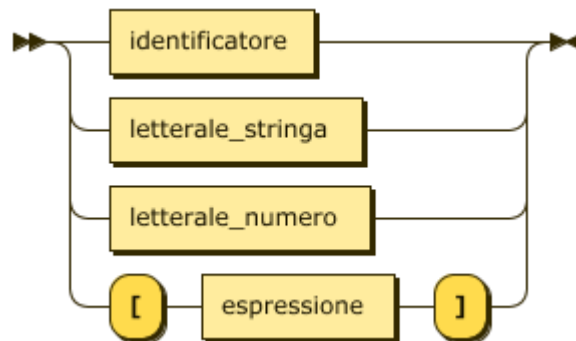


def_proprietà:



```
letterale_oggetto ::= '{' lista_proprietà '}'
lista_proprietà ::= | def_proprietà ',' lista_proprietà
def_proprietà ::= identificatore | chiave ':' espressione
chiave ::= identificatore
           | letterale_stringa
           | letterale_numero
           | '[' espressione ']'
```

chiave:



Alcuni esempi di letterali di tipo oggetto

`{ }` – oggetto vuoto

`{ nome: “Pippo”, età: 35 }` – il caso più comune

`{ x }` – *espansione di variabile*: se *x* vale 5, è come `{ x: 5 }`

`{ “nome”: “Pippo”, “età” : 35 }` – le chiavi possono essere stringhe

`{ “!!”: 5, “🕶️”: true }` – anche stringhe che non sono identificatori

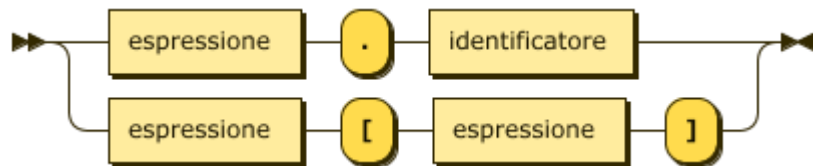
`{ 0: 6, 1: 4, 2: 12 }` – le chiavi possono essere numeri

`{ [ “pip”+fiume ]: true }` – *chiave calcolata*: se *fiume* vale “po”, è come `{ pippo: true }`

Operazioni sugli oggetti

L'operazione principale sugli oggetti è l'**accesso** a un membro (chiave)

espressione_accesso:



*espressione_accesso ::= espressione '.' identificatore
| espressione '[' espressione ']'*

```
let pippo = { nome: "Filippo", età: 35, capelli: true }
```

```
pippo.nome → "Filippo"
```

```
pippo["nome"] → "Filippo"
```

```
pippo.età → 35
```

```
if (pippo.capelli) {  
    pippo.età++  
    pippo.nome="Guendalino"  
}
```

```
pippo → { capelli: true, età: 36, nome: "Guendalino" }
```

```
pippo.occhi → undefined
```

```
pippo.occhi = 2
```

```
pippo → { capelli: true, età:  
36, nome: "Guendalino",  
occhi: 2 }
```


Operazioni sugli oggetti

Altre operazioni utili:

- Controllo se una chiave esiste: *chiave in oggetto*
- Cancellazione di una coppia chiave-valore: **delete** *chiave*
- Scorrere le chiavi: **for** (*k in oggetto*) ...

```
let pippo = { nome: "Filippo", età: 35, capelli: true }
```

```
"nome" in pippo → true
```

```
"occhi" in pippo → false
```

```
delete pippo.nome
```

```
pippo → { capelli: true, età: 35 }
```

```
for (var k in pippo)
```

```
    delete pippo[k]
```

```
pippo → { }
```

Insiemi

Usando gli oggetti possiamo definire degli insiemi

Le *chiavi* dell'oggetto sono gli elementi dell'insieme

I *valori* diversi da undefined indicano gli elementi presenti

Possiamo definire funzioni che fanno delle operazioni su insiemi.

```
let A = { mele: 1, pere: 1, arance: 1 }  
let B = { arance: 1, pere: 1, prugne: 1 }  
let c= intersezione(A,B)
```

```
function intersezione(a,b){  
  var c={};  
  for (e in a)  
    if (e in b)  
      c[e]=1;  
  return c;  
}
```

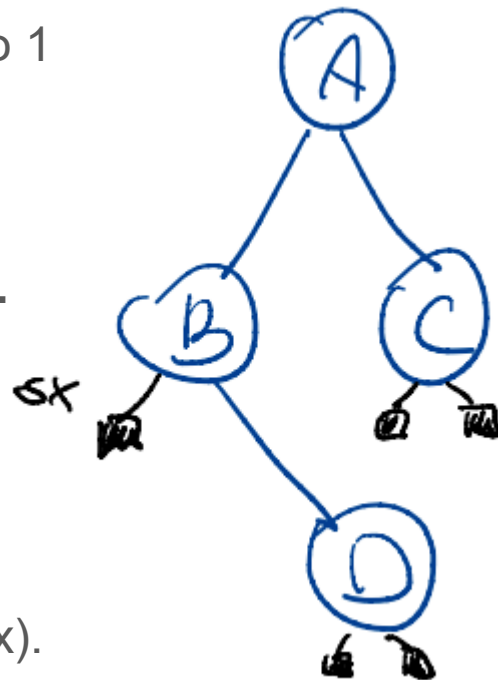
Alberi

Un **albero** è un **grafo connesso aciclico**. I nodi di grado 1 sono detti **foglie**, i nodi di grado > 1 sono detti **nodi interni**.

Un **albero radicato** ha un nodo identificato come **radice**.

Ogni nodo interno ha 1 o più **figli**, che consideriamo ordinati (e numerati $0 \dots k$).

Nel caso particolare di $k=2$, si parla di **albero binario**; in questo caso, chiameremo i figli **sinistro** (sx) e **destro** (dx).



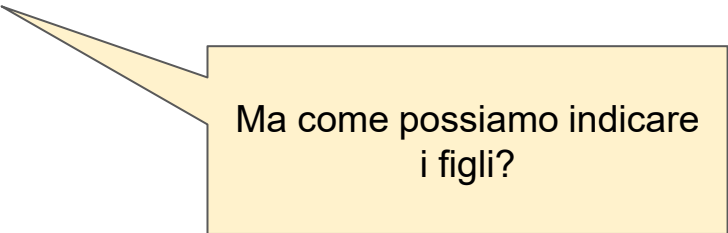
Codifica degli alberi

Concentriamoci sugli alberi binari, e associamo ad ogni nodo un qualche **valore**.

Possiamo allora rappresentare un nodo con un oggetto JavaScript con tre chiavi

- **val**: il valore associato al nodo
- **sx**: il figlio sinistro del nodo (chiave assente o **null** se il figlio sx manca)
- **dx**: il figlio destro del nodo (chiave assente o **null** se il figlio dx manca)

```
{ val: 5, sx: ..., dx: ... }
```



Ma come possiamo indicare i figli?

Codifica degli alberi

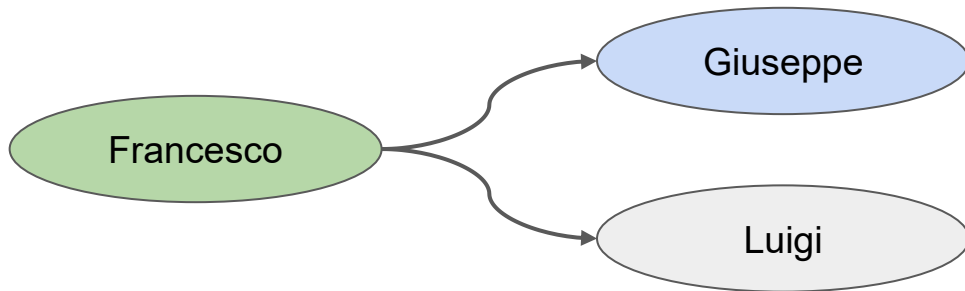
I figli di un nodo sono anch'essi nodi... quindi usiamo la stessa codifica!

```
let pino={val: "Giuseppe"}, gigi={val: "Luigi"}
```

```
let ciccio={val: "Francesco", sx: pino, dx: gigi}
```

Oppure, direttamente:

```
let ciccio={val: "Francesco", sx: {val: "Giuseppe"}, dx:  
{val: "Luigi"} }
```

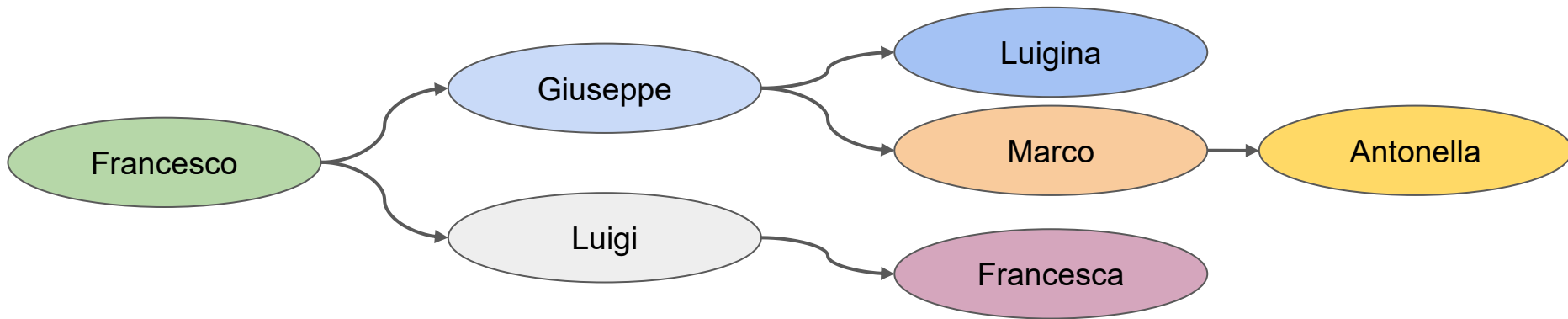


Codifica degli alberi

Gli alberi sono naturalmente una struttura **ricorsiva**.

Abbiamo nodi, i cui figli sono nodi, i cui figli sono nodi ... fino ad arrivare a nodi senza figli.

Se sappiamo come manipolare un nodo e i suoi figli per fare qualcosa... allora sappiamo come manipolare un albero intero, di qualunque dimensione!



Alberi k -ari a valori numerici

Ora che sappiamo manipolare gli alberi binari (o 2-ari), possiamo spingerci oltre e considerare gli alberi k -ari, con $k > 2$.

Non possiamo più dare un *nome* ai figli (dx e sx)... ora potremmo avere un numero qualunque di figli.

Fortunatamente, JavaScript ha giusto un tipo di dato che fa al caso nostro:

```
{ val: 5, figli: [ ... ] }
```

Alberi k -ari a valori arbitrari

Possiamo sfruttare il fatto che il valore associato a un nodo può essere di qualunque tipo (anche diverso da nodo a nodo) per creare strutture interessanti

```
{ val: " ... ", figli: [ ... ] }
```

```
{ val: { ... }, figli: [ ... ] }
```

```
{ val: [ ... ], figli: [ ... ] }
```


Array

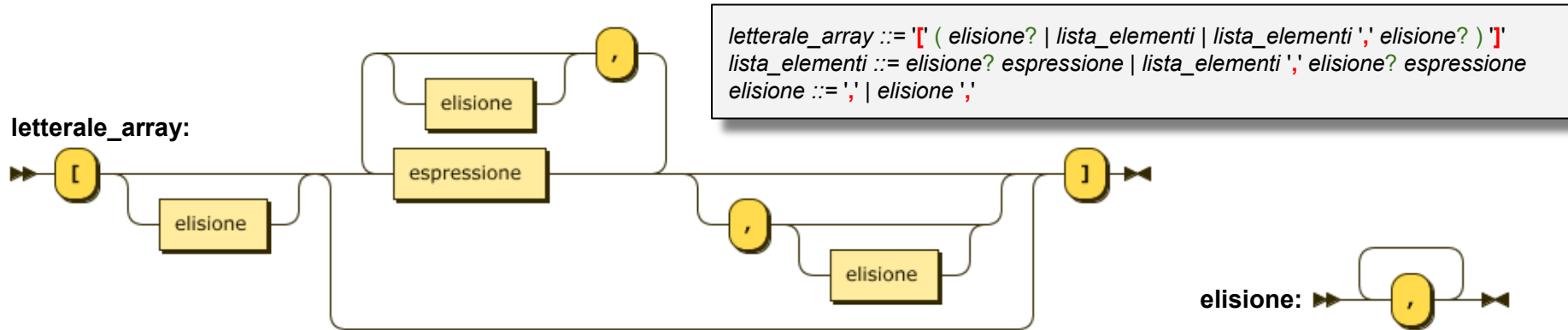
Gli **array** sono un tipo particolare di oggetto, caratterizzato dal fatto che le chiavi sono **numeri interi**

In Informatica, si amano in particolare gli array le cui chiavi sono tutti i numeri interi fra 0 e un numero fissato n , e i cui valori sono tutti dello stesso tipo

Nessuna di queste condizioni è però necessaria in JavaScript!

L'ordinamento dei numeri naturali (le chiavi) definisce un ordinamento fra le coppie (chiave, valore): gli array si possono quindi considerare **liste** (ovvero: sequenze ordinate) anziché **insiemi**

Costanti letterali di tipo array - Sintassi



In pratica, è un modo un po' complicato di dire: `[ espressione , espressione , ... ]`, in cui però una o più delle *espressioni* possono mancare.

Le espressioni mancanti occupano sempre un "posto" nell'array, con il loro indice, ma non hanno un valore. Quelle mancanti in fondo all'array vengono invece ignorate.

```
let pluto = [ , 10 , , 4 , ]

pluto → [ empty, 10, empty, 4 ]
```

Operazioni sugli array - accesso

Nel caso degli array, specialmente se “tradizionali”, si usa chiamare le chiavi **indici** e i valori **elementi** dell’array.

Poiché le chiavi sono numeri, e quindi non sono identificatori, l’unico operatore di accesso possibile è quello con le parentesi quadre. In compenso, esiste una notazione più comoda per le costanti letterali di tipo array.

chiave indice	valore elemento
0	“Jacopo”
1	“Vincenzo”
2	“Giuseppe”
3	“Davide”

Array di lunghezza 4
(indici fra 0 e 3)

```
var a = [ "Jacopo", "Vincenzo", "Giuseppe", "Davide" ]
```

```
a[0] → "Jacopo"
```

```
a[2]="Peppe"
```

```
a → [ "Jacopo", "Vincenzo", "Peppe", "Davide" ]
```

```
a.length → 4
```

Altre operazioni sugli array

Molte altre operazioni anche più interessanti, ispirate alla **programmazione funzionale!**

Gli array consentono ovviamente tutte le operazioni consentite sugli object.

In più, hanno una serie di funzioni speciali che semplificano le operazioni su liste.

- `a.push(z)` – aggiunge `z` in coda all'array `a`
- `c=a.concat(b)` – concatena l'array `b` in coda all'array `a`, restituisce il risultato
- `z=a.pop()` – rimuove l'ultimo elemento da `a`, restituisce l'elemento rimosso
- `x=a.shift()` – rimuove il primo elemento di `a`, restituisce l'elemento rimosso; tutti gli altri elementi scorrono di un posto per “chiudere il buco”
- `a.sort()` – ordina un array in base al valore degli elementi (possiamo passare un criterio di ordinamento)
 - Ma non ditelo a quelli di Programmazione & Algoritmica!

Object e array

Gli array sono un tipo particolare di oggetti, **ma** non sono *esattamente* la stessa cosa

```
var a = [ "Jacopo", "Vincenzo", "Giuseppe", "Davide" ]  
var b = { 0: "Jacopo", 1: "Vincenzo", 2: "Giuseppe", 3: "Davide" }  
  
a==b → false  
  
for (k in a)  
    console.log(a[k]==b[k]) → stampa 4 volte true  
  
for (k in b)  
    console.log(a[k]==b[k]) → stampa 4 volte true  
  
typeof a → "object"  
typeof b → "object"
```

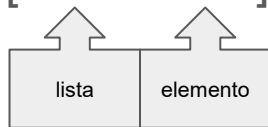
La differenza sta in effetti nel **tipo**, ma in un senso più profondo di quanto possiamo discutere adesso.

Array come **liste**

Gli array possono essere usati anche come **liste**, ovvero come **sequenze** di elementi di lunghezza variabile.

In questo stile, raramente si accede agli elementi tramite la loro posizione (*per indice*). Invece, si lavora in maniera **strutturale**:

- Una **lista vuota** è una lista: `[ ]`
- Un **elemento seguito da una lista** è una lista: `[testa resto]`
- In maniera analoga, ci si può concentrare sull'ultimo elemento: `[resto coda]`



**Definizioni
ricorsive!**

Array come liste

Alcuni metodi predefiniti consentono di scorrere un array (come lista) in maniera simile a quanto abbiamo visto negli esempi, ma più generale.

Se A è un array di elementi di tipo E , $p:E \rightarrow \text{Bool}$ un predicato, $f:E \rightarrow X$ una funzione

A.every(p)	Restituisce true se tutti gli elementi di A soddisfano il predicato p , false altrimenti
A.some(p)	Restituisce true se almeno uno degli elementi di A soddisfa il predicato p , false altrimenti
A.find(p)	Restituisce un elemento e di A tale che $p(e)$ è true, o undefined se nessun e soddisfa p
A.findIndex(p)	Restituisce l'indice di un elemento e di A tale che $p(e)$ è true, o -1 se nessun e soddisfa p
A.includes(e)	Restituisce true se A contiene un elemento uguale a e , false altrimenti
A.forEach(f)	Invoca $f(e)$ per ogni elemento e dell'array A
A.map(f)	Restituisce un nuovo array $A' = [f(e_1), f(e_2), \dots f(e_n)]$
A.filter(p)	Restituisce un nuovo array A' contenente i soli elementi e di A che soddisfano p , in ordine

Array come liste

Sono a volte utili le funzioni di **riduzione**.

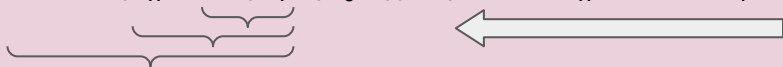
L'idea è semplice: si parte da un valore dato che costituisce il risultato iniziale, poi si scorre la lista, un elemento alla volta, e si applica una funzione data al risultato precedente e al nuovo elemento. Alla fine della lista, si restituisce il risultato.

A.reduce(f,z)	Fa una riduzione da sinistra a destra, partendo da z, e applicando f
A.reduceRight(f,z)	Fa una riduzione da destra a sinistra, partendo da z, e applicando f

Esempio

A.reduce((r,e)=>r+e, 0) — restituisce la somma di tutti gli elementi di A

$$f(e_n, \dots f(e_1, f(e_0, z)) \dots) \text{ — } e_n + \dots + e_1 + e_0 + 0$$



Assegnamenti destrutturanti

Il concetto di *destrutturazione* è semplice: si prende un **tipo strutturato** (array, oggetto) e lo si “spacchetta” andando a guardare al suo interno.

In molti casi in cui normalmente è previsto un **nome di variabile** (tecnicamente: una *left-hand side* o lhs, ciò che può stare a sinistra di un =), JavaScript consente di usare la notazione di un **letterale** array o oggetto, in cui però al posto dei valori si indicano dei **nomi di variabili** (o meglio: dei lhs).

Notate che anche questa definizione è ricorsiva: si possono destrutturare array o oggetti che contengono altri array o oggetti come elementi, e così via ...

Assegnamenti destrutturanti

Alcuni esempi sugli array

Se $A = [4, 7, 1]$:

- $[a, b] = A$ $a \rightarrow 4$ $b \rightarrow 7$
- $[a, b, c, d] = A$ $a \rightarrow 4$ $b \rightarrow 7$ $c \rightarrow 1$ $d \rightarrow \text{undefined}$
- $[a, b, c=3, d=8] = A$ $a \rightarrow 4$ $b \rightarrow 7$ $c \rightarrow 1$ $d \rightarrow 8$
- $[, , c] = A$ $c \rightarrow 1$
- $[a, ...r] = A$ $a \rightarrow 4$ $r \rightarrow [7, 1]$
- $[y, x] = [x, y]$ scambia i valori di x e y (senza variabili
intermedie o XOR!)
- $[x, y] = f(\text{"pippo"}, 3)$ ottiene due valori di ritorno distinti da una funzione

Lo spread

L'operatore di spread è denotato da tre punti consecutivi: ...

Ha il significato generale di trasformare una *sequenza di elementi singoli* in un *singolo dato strutturato* e viceversa.

Alcuni esempi:

- $f(\dots A)$ — chiama la funzione f , passando gli elementi di A come argomenti singoli
- $p = \{ \dots q, c: 3 \}$ — p è una copia dell'oggetto q , con la chiave c che vale 3
- $p = \{ c: 3, \dots q \}$ — p è una copia di q , con aggiunto $c=3$ se manca in q , altrimenti il valore di c in q
- $B = [1, 2, \dots A, 6]$ — B è un array contenente gli elementi di A , ma con 1 e 2 davanti e 6 in coda
- $C = [\dots A, \dots B]$ — C è la concatenazione degli array A e B
- $R = \{ \dots O1, \dots O2 \}$ — R è l'unione dei due oggetti $O1$ e $O2$; se hanno delle chiavi in comune, “vince” il valore di $O2$
- $[\text{testa}, \dots \text{resto}] = A$ — testa prende il primo elemento di A , resto tutti gli altri



Phew!